

Software Requirements Specification

For Face Recognition-Based Smart Attendance Management System with Analytics and Reporting

Project ID: 20

Version 1.0 (Approved)

UE24CS341A - Software Engineering, Jackfruit Mini-Project (Phase 1)

Team Details

Field	Details
Team Name	SE GROUP
Team Member 1	MITHUN S SRN: PES2UG24AM208
Team Member 2	BHARATH H G SRN: PES2UG24AM197
Team Member 3	VARUN C H SRN: PES2UG24AM180
Team Member 4	YASHWANT SRN: PES2UG24AM190

Prepared by MITHUN S

AIML 5C

05/09/2026

Table of Contents

Team Details	2
Revision History	Error! Bookmark not defined.
Table of Contents.....	3
1. Introduction	5
1.1 Purpose.....	5
1.2 Intended Audience and Reading Suggestions	5
1.3 Product Scope	5
1.4 References	5
2. Overall Description	5
2.1 Product Perspective.....	5
2.2 Product Functions	5
2.3 User Classes and Characteristics	6
2.4 Operating Environment.....	6
2.5 Design and Implementation Constraints	6
2.6 Assumptions and Dependencies	6
3. External Interface Requirements.....	7
3.1 User Interfaces.....	7
3.2 Software Interfaces.....	7
3.3 Communications Interfaces	7
3.4 Hardware Interfaces	7
4. Analysis Models	7
5. System Features.....	8
5.1 Student Face Enrollment	8
5.1.1 Description and Priority	8
5.1.2 Stimulus/Response Sequences	8
5.1.3 Functional Requirements.....	8
5.2 Automated Attendance Capture (Face Recognition).....	8
5.2.1 Description and Priority	8
5.2.2 Stimulus/Response Sequences	9
5.2.3 Functional Requirements.....	9
5.3 Manual Attendance Override.....	9
5.3.1 Description and Priority	9
5.3.2 Stimulus/Response Sequences	9
5.3.3 Functional Requirements.....	9
5.4 Attendance Analytics Dashboard.....	9
5.4.1 Description and Priority	9
5.4.2 Stimulus/Response Sequences	9
5.4.3 Functional Requirements.....	9
5.5 Report Generation and Export	10
5.5.1 Description and Priority	10

5.5.2 Stimulus/Response Sequences	10
5.5.3 Functional Requirements	10
5.6 User Authentication and Role Management	10
5.6.1 Description and Priority	10
5.6.2 Stimulus/Response Sequences	10
5.6.3 Functional Requirements	10
6. Other Nonfunctional Requirements	10
6.1 Performance Requirements	10
6.2 Safety Requirements	11
6.3 Security Requirements	11
6.4 Software Quality Attributes	11
6.5 Business Rules	11
7. Other Requirements	11
Appendix A: Glossary	11
Appendix B: Field Layouts	12
B.1 Student	12
B.2 Attendance	12
Appendix C: Requirement Traceability Matrix	12

(Right-click the table above and choose “Update Field” in Word to populate page numbers.)

1. Introduction

1.1 Purpose

This document specifies the software requirements for the “Face Recognition-Based Smart Attendance Management System with Analytics and Reporting” (hereafter “the System”), Version 1.0. The System automates classroom/lab attendance capture using facial recognition and provides role-based dashboards, analytics, and exportable reports for faculty and administrators. This SRS covers the complete system: enrollment, recognition-based attendance marking, manual override, analytics, and reporting.

1.2 Intended Audience and Reading Suggestions

This document is intended for the development team (for design and implementation), the course faculty/evaluators (for review and grading), and future maintainers of the project. Section 2 gives a high-level overview; Section 3 covers interfaces; Section 4 shows the use-case model; Section 5 lists detailed functional requirements by feature; Section 6 lists non-functional requirements. Readers interested only in scope should read Sections 1 and 2; readers implementing a specific feature should read the corresponding subsection of Section 5.

1.3 Product Scope

The System replaces manual, paper/roll-call-based attendance with an automated pipeline: a camera captures a frame of the classroom, faces are detected and matched against enrolled student face embeddings, and matched students are marked present for that session automatically. Faculty can review and correct the auto-marked attendance before it is finalised. Admins manage student/faculty accounts and course-section mappings. The system generates attendance analytics (per-student, per-section, per-date-range) and exports reports as CSV/PDF. Goals: reduce time spent on manual roll-call, reduce proxy attendance, and give faculty/students visibility into attendance trends. Out of scope for this mini-project: biometric fingerprint/RFID hardware integration, mobile native apps (a responsive web UI is used instead), and campus-wide multi-building deployment.

1.4 References

- IEEE Std 830-1998 – SRS Template (as provided in the Jackfruit SE Mini-Project Guidelines, 2026)
- Jackfruit – SE Mini-Project Guidelines 2026, Dept. of CSE (AI/ML), PES University EC Campus
- face_recognition (Python library, built on dlib) – https://github.com/ageitgey/face_recognition
- OpenCV Documentation – <https://docs.opencv.org/>
- Flask Documentation – <https://flask.palletsprojects.com/>

2. Overall Description

2.1 Product Perspective

The System is a new, self-contained web application; it is not a follow-on to an existing product. It consists of three logical components: (a) a Flask-based web application serving role-based dashboards (Admin, Faculty, Student); (b) a face-recognition service (Python, OpenCV + face_recognition/dlib) that performs face detection and embedding-matching against a stored face database; and (c) a relational database (SQLite for development, MySQL/PostgreSQL for deployment) storing users, enrolled face encodings, sessions, and attendance records. The web app and the recognition service run in the same Python process for the mini-project scope, communicating through a shared service layer rather than a network API.

2.2 Product Functions

- Enroll a student's face (capture a few reference images, generate and store a face encoding)
- Automatically mark attendance for a class session by matching faces detected in a captured photo/video frame against enrolled encodings
- Allow faculty to manually override/correct auto-marked attendance for a session
- Provide an analytics dashboard: attendance % per student, per section, and over a date range, with charts
- Generate and export attendance reports (CSV/PDF) for a course, section, or date range
- Role-based authentication and management of Admin, Faculty, and Student accounts

2.3 User Classes and Characteristics

User Class	Characteristics	Key Functions Used
Admin	Highest privilege; typically one per department. Comfortable with basic web forms; not necessarily technical.	Manage users, sections, and course mappings; view all analytics; system configuration
Faculty	Takes attendance in class; uses the system every lecture. Moderate frequency of use, low technical expertise expected.	Start attendance session, review/override auto-marked attendance, view section analytics, export reports
Student	Lowest privilege, occasional use (checking own record). Minimal technical expertise assumed.	View own attendance record and percentage; view enrollment status

2.4 Operating Environment

- Server: any machine capable of running Python 3.10+; developed and tested on Windows/Linux.
- Client: any modern web browser (Chrome, Edge, Firefox) with a minimum screen width of 360px for the responsive UI.
- Camera: a standard USB/laptop webcam (minimum 720p recommended) attached to the faculty/admin machine used to capture attendance frames.
- Database: SQLite for local development; MySQL 8+/PostgreSQL 13+ for a shared deployment.

2.5 Design and Implementation Constraints

- Must be implemented in Python using Flask (per team's chosen stack) for the web layer.
- Face recognition must use an open-source, offline-capable library (e.g., face_recognition/dlib or OpenCV DNN face module) – no paid third-party face-recognition API, to keep the project free to run and demo.
- Face images/encodings are personal biometric data and must not be committed to the public GitHub repository in raw form; only hashed/encoded vectors are stored in the database, and sample/demo images used for testing must be synthetic or consented.
- Must follow Agile methodology with a GitHub-tracked backlog (issues/project board) as mandated by the course guidelines.

2.6 Assumptions and Dependencies

- Assumes reasonably well-lit classrooms/labs and that students face the camera at the start of class for capture; extreme lighting/occlusion (masks, hats covering the face) may reduce recognition accuracy.
- Assumes each student enrolls at least 3-5 reference face images at the start of the course.
- Depends on third-party open-source libraries (OpenCV, dlib/face_recognition, Flask, SQLAlchemy) remaining available and compatible with the chosen Python version.
- Assumes one camera is sufficient per classroom capture (no multi-camera stitching in this version).

3. External Interface Requirements

3.1 User Interfaces

The system provides a responsive, browser-based UI with three role-specific dashboards (Admin, Faculty, Student), reached after login. Common elements across all screens: a top navigation bar showing the logged-in user's name and role, a logout button, and consistent primary-action buttons (blue, top-right of each panel). Error messages are shown as a dismissible red banner at the top of the form; success messages as a green banner. The Faculty “Take Attendance” screen shows a live camera preview, a “Capture” button, and, after processing, a list of recognised students with checkboxes so faculty can review/uncheck before saving. Screen wireframes/mockups are to be added by the team in the SW Design document (Phase 2), following standard Bootstrap UI conventions.

3.2 Software Interfaces

Component	Name & Version	Purpose
Web framework	Flask 3.x (Python)	Serves web pages, REST-style routes, session/auth handling
Face detection/recognition	face_recognition (dlib-based) or OpenCV 4.x DNN face module	Detects faces in a frame and computes/matches 128-d face encodings
ORM / Database driver	SQLAlchemy + SQLite (dev) / MySQL or PostgreSQL (prod)	Persists users, encodings, sessions, attendance records
Frontend	HTML5, Bootstrap 5, Chart.js	Responsive UI and analytics charts
Report generation	pandas + reportlab / WeasyPrint	CSV and PDF report export

3.3 Communications Interfaces

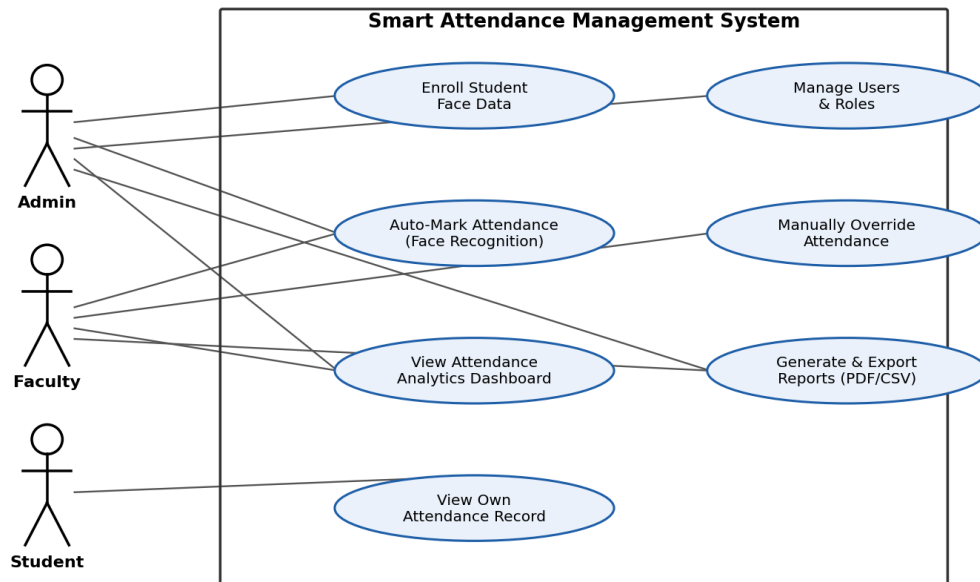
All client-server communication uses HTTPS over standard TCP/IP on the deployment network (HTTP is acceptable for local development only). The browser communicates with the Flask server using standard HTTP requests/responses (form submissions and JSON for AJAX-driven dashboard updates). No external network APIs are required; the system does not send email/SMS in this version (see Section 7 for a possible future notification interface).

3.4 Hardware Interfaces

The only hardware interface is a standard UVC-compliant USB/integrated webcam accessed through the OS's camera driver via OpenCV's VideoCapture API on the machine running the Faculty/Admin “Take Attendance” screen. No other special-purpose hardware is required.

4. Analysis Models

The use-case diagram below summarises the actors and the primary use cases of the system.



Primary actors: Admin (manages users/sections, sees all data), Faculty (takes and corrects attendance, views section analytics), Student (views own record only). An Entity-Relationship model (User, Student, Face_Encoding, Section, Session, Attendance) is to be added by the team in the SW Design document (Phase 2), once the schema in Appendix B is finalised.

5. System Features

5.1 Student Face Enrollment

5.1.1 Description and Priority

Allows an Admin/Faculty to capture reference face images for a student and store the resulting face encoding against the student's profile. Priority: High – all other attendance features depend on this.

5.1.2 Stimulus/Response Sequences

Admin/Faculty selects a student → opens “Enroll Face” → system activates camera preview → user captures 3-5 images → system detects a face in each, computes an encoding, and shows a success/failure confirmation → system stores the encoding against the student record.

5.1.3 Functional Requirements

REQ-EN-1: The system shall allow capture of at least 3 reference face images per student via the connected webcam.

REQ-EN-2: The system shall reject an image if no face, or more than one face, is detected in it, and shall prompt the user to recapture.

REQ-EN-3: The system shall compute and store a face encoding vector (not the raw image) linked to the student's unique ID.

REQ-EN-4: The system shall allow an Admin/Faculty to re-enroll (overwrite) a student's face encoding.

5.2 Automated Attendance Capture (Face Recognition)

5.2.1 Description and Priority

Captures a classroom photo/frame at the start of a session and automatically marks all recognised, enrolled students as present. Priority: High – this is the core feature of the project.

5.2.2 Stimulus/Response Sequences

Faculty selects a section → clicks “Start Attendance Session” → system opens camera preview → faculty clicks “Capture” → system detects all faces in the frame, matches each against enrolled encodings for that section, and displays the list of recognised students → faculty reviews and clicks “Save Attendance”.

5.2.3 Functional Requirements

REQ-AT-1: The system shall detect all faces present in a captured frame.

REQ-AT-2: The system shall match each detected face against the enrolled encodings of students registered in the selected section, using a configurable similarity threshold.

REQ-AT-3: The system shall mark a student “Present” for the session if a match exceeds the threshold, and “Absent” otherwise.

REQ-AT-4: The system shall record the session timestamp, section, and captured-frame reference alongside the attendance result.

REQ-AT-5: If no face in the frame matches a given enrolled student above the threshold, the system shall leave that student marked absent rather than guessing the closest match.

5.3 Manual Attendance Override

5.3.1 Description and Priority

Lets faculty correct the auto-marked attendance (e.g., a misrecognised or undetected student) before it is finalised.

Priority: High – required to keep the system trustworthy despite recognition errors.

5.3.2 Stimulus/Response Sequences

After auto-marking, faculty sees a checklist of students with Present/Absent toggles → faculty changes any incorrect entries → clicks “Save Attendance” → system persists the final, faculty-confirmed record.

5.3.3 Functional Requirements

REQ-OV-1: The system shall display the auto-marked attendance list to faculty before it is finalised, with an editable Present/Absent control per student.

REQ-OV-2: The system shall allow faculty to change any student's status and shall log the change (who changed it, and when) for audit purposes.

REQ-OV-3: The system shall not allow a Student-role user to override attendance.

5.4 Attendance Analytics Dashboard

5.4.1 Description and Priority

Provides visual summaries of attendance (percentage present, trend over time) for a student, a section, or a date range.

Priority: Medium.

5.4.2 Stimulus/Response Sequences

Faculty/Admin selects a section and date range → system queries attendance records → system renders a bar/line chart of attendance % and a sortable table of individual student percentages; a Student sees only their own chart on login.

5.4.3 Functional Requirements

REQ-AN-1: The system shall compute each student's attendance percentage as (sessions present / sessions held) x 100 for a selected section and date range.

REQ-AN-2: The system shall display a chart of attendance percentage trends over time for a selected section.

REQ-AN-3: The system shall flag (e.g., highlight in red) any student whose attendance percentage falls below a configurable threshold (default 75%).

REQ-AN-4: A Student-role user shall only be able to view analytics for their own record.

5.5 Report Generation and Export

5.5.1 Description and Priority

Allows faculty/admin to export attendance data as CSV or PDF for record-keeping or submission to the department.

Priority: Medium.

5.5.2 Stimulus/Response Sequences

Faculty/Admin selects a section and date range on the Reports screen → clicks “Export CSV” or “Export PDF” → system generates the file and offers it as a download.

5.5.3 Functional Requirements

REQ-RP-1: The system shall export attendance records for a selected section and date range as a CSV file.

REQ-RP-2: The system shall export a formatted attendance summary (including attendance %) as a PDF file.

REQ-RP-3: Exported reports shall include section name, date range, and generation timestamp in the file header.

5.6 User Authentication and Role Management

5.6.1 Description and Priority

Provides login and role-based access control for Admin, Faculty, and Student users. Priority: High – required before any other feature can be safely used.

5.6.2 Stimulus/Response Sequences

User enters username/email and password on the login screen → system validates credentials → on success, redirects to the role-appropriate dashboard; on failure, shows an error message.

5.6.3 Functional Requirements

REQ-AU-1: The system shall authenticate users via a username/email and password before granting access to any dashboard.

REQ-AU-2: The system shall store passwords as salted hashes, never in plaintext.

REQ-AU-3: The system shall restrict each screen/action to the roles authorised for it, as defined in Section 2.3.

REQ-AU-4: The system shall allow an Admin to create, deactivate, and reset the password of Faculty and Student accounts.

6. Other Nonfunctional Requirements

6.1 Performance Requirements

- Face detection and matching for a captured classroom frame (up to ~60 students) shall complete within 5 seconds on a standard laptop CPU (no GPU assumed).

- Dashboard pages (analytics, reports) shall load within 3 seconds for a section of up to 100 students and 1 academic year of session data.

6.2 Safety Requirements

Not applicable in the physical-safety sense (the system is informational software, not a controller of physical equipment). The main “safety” concern is data integrity: an attendance-marking failure must never silently mark a present student absent without faculty being able to review and correct it (see REQ-OV-1).

6.3 Security Requirements

- Face encodings and personal data shall be stored in the application database only, never committed to the GitHub repository.
- All communication in a deployed (non-local) environment shall use HTTPS.
- Role-based access control (Section 2.3) shall be enforced on every server-side route, not just hidden in the UI.
- Session tokens/cookies shall expire after a configurable period of inactivity (default 30 minutes).

6.4 Software Quality Attributes

- Usability: a faculty member should be able to complete a full attendance-taking cycle (capture → review → save) in under 1 minute for a class of 60.
- Maintainability: the recognition/matching logic shall be isolated in its own module/service layer, independent of Flask route handlers, so the matching algorithm can be swapped without touching the UI code.
- Reliability: if face recognition fails or the camera is unavailable, the system shall fall back to allowing fully manual attendance entry rather than blocking the session.
- Testability: each functional requirement in Section 5 shall have at least one corresponding automated or manual test case in the Phase-3 Test Plan.

6.5 Business Rules

- Only Faculty assigned to a section may start an attendance session for that section.
- Only one attendance record may exist per student per session; re-capturing a session updates the existing record rather than creating a duplicate.
- A student below the configured attendance threshold (default 75%) shall be visibly flagged to both Faculty and the Student themselves.

7. Other Requirements

Internationalization: not required for this version (English-only UI). Legal/compliance: because the system processes biometric (facial) data, the team should note applicable institutional data-handling guidance in the final report and obtain simple consent from classmates before enrolling their faces for demo/testing purposes. Future scope (not built in this version, listed for completeness): SMS/email low-attendance notifications to students, and a native mobile app for the Student view.

Appendix A: Glossary

Term	Definition
Face Encoding	A fixed-length numeric vector (embedding) representing a face, used to compare two faces for similarity instead of comparing raw pixels.

Term	Definition
Session	One instance of a class/lab meeting for which attendance is taken (e.g., “CS301 – 3 Sept, 9:00 AM”).
Section	A group of students taking a course together (e.g., “Section A”).
Recognition Threshold	The minimum similarity score between a detected face and a stored encoding for the system to accept it as a match.
RTM	Requirement Traceability Matrix – maps each requirement to its design, code, and test references (Appendix C).

Appendix B: Field Layouts

Core data fields for the two central tables. The full schema (including Section, Session, and User/role tables) is to be finalised by the team in the Phase-2 design document.

B.1 Student

Field	Length	Data Type	Description	Mandatory
student_id	10	Alphanumeric	Unique student identifier / SRN	Y
name	100	Text	Full name	Y
section_id	10	Alphanumeric	Foreign key to Section	Y
face_encoding	-	Binary/Blob	Serialized 128-d face encoding vector	N (set on enrollment)
email	100	Text	Contact email	N

B.2 Attendance

Field	Length	Data Type	Description	Mandatory
attendance_id	10	Numeric	Unique record identifier	Y
student_id	10	Alphanumeric	Foreign key to Student	Y
session_id	10	Alphanumeric	Foreign key to Session	Y
status	1	Enum (P/A)	Present or Absent	Y
marked_by	10	Alphanumeric	“auto” or faculty user ID if manually overridden	Y
timestamp	20	DateTime	When the record was created/last updated	Y

Appendix C: Requirement Traceability Matrix

To be populated as design and test artefacts are produced in Phases 2 and 3; the structure below follows the course template.

Sl. No	Requirement ID	Brief Description	Architecture Ref.	Design Ref.	Code File Ref.	Test Case ID	System Test Case ID